SignalDNA: Reproducible Neural-Network Training and CPU/GPU Performance Characterization

David Boring | Independent research | October 2026

Status: Completed initial experimental study; manuscript under review. Not peer reviewed.

ABSTRACT

SignalDNA investigates reproducible neural-network training, generalization, and computational performance through synthetic transient detection. A 67,713-parameter feedforward neural network was implemented from first principles in NumPy and compared with a PyTorch CUDA training implementation. CPU and CUDA learning metrics agreed across 61 recorded checkpoints, with a maximum absolute difference of approximately 2.22e-16. The locked epoch-60 model achieved 78.78% accuracy and 0.8613 ROC-AUC in one authorized held-out evaluation of 1,800 waveforms. In the measured reference workload, CPU training was faster than CUDA. The study documents successful numerical replication, meaningful classification errors, a substantial generalization gap, and workload-specific computational trade-offs. Its broader purpose is to establish a foundation for future neural-network investigations in embedded systems and FPGA engineering.

1. Introduction

Neural networks may offer useful approaches to signal interpretation, hardware testing, design assistance, and engineering analysis. SignalDNA was undertaken to explore these possibilities by first understanding the underlying computational methods.

Instead of beginning with a pretrained neural network, the project developed a NumPy reference implementation with explicit forward propagation, backpropagation, and parameter updates. GPU acceleration was investigated after establishing and verifying this mathematical foundation.

The research questions concern numerical reproducibility, held-out generalization on synthetic signals, and CPU/GPU performance under controlled implementation comparisons.

1.1 Research motivation and personal learning objectives

SignalDNA began as a practical exploration of neural networks and their potential relevance to my work in embedded systems and FPGA development. My long-term objective is to investigate whether these methods can meaningfully assist with hardware design, verification, testing, and engineering problem-solving in future projects.

Rather than beginning with a pretrained model, I chose to implement a neural network from first principles. This allowed me to examine data representation, forward propagation, backpropagation, optimization, and evaluation before exploring hardware-accelerated implementations.

The project has been both an educational experiment and an enjoyable engineering challenge. It has provided a clearer understanding of neural-network behavior, controlled experimentation, and CPU/GPU computational trade-offs. It has also made clear how much remains to be learned.

SignalDNA is an early step toward a broader machine-learning skill set, not a demonstration that neural networks already improve FPGA or embedded engineering workflows. Future work will investigate those applications through measurable engineering outcomes, hardware constraints, reproducibility, and practical usefulness.

2. Methods

Dataset construction

SignalDNA uses a deterministic synthetic binary-classification dataset containing 12,000 waveforms. Each waveform comprises 1,024 float64 samples at a sampling frequency of 2,048 Hz, corresponding to a duration of 0.5 seconds.

Each dataset split is exactly class-balanced. Class 0 contains background waveforms; class 1 contains background waveforms with an added localized Gaussian transient. The splits contain 8,400 training examples, 1,800 validation examples, and 1,800 held-out test examples.

Separate pseudorandom streams are derived using NumPy SeedSequence.spawn(3), with parent seed 2026 [2]. This separates the streams but does not mathematically guarantee waveform uniqueness. The held-out test set was evaluated once after the checkpoint selection and evaluation protocol were locked.

Background waveform generation

The fundamental component is:

```
x_base(t) = A sin(2*pi*f*t + phi)
```

where f is sampled uniformly from 20 to 200 Hz, A from 0.5 to 1.5, and phi from 0 to 2*pi.

Zero to three optional harmonics, corresponding to orders 2 through 4, are added. Each harmonic has a strength sampled uniformly from 0 to 0.5, scaled by the fundamental amplitude. In the implemented generator, harmonic phases are zero rather than inherited from the fundamental phase.

Independent Gaussian noise is added with standard deviation A*r, where r is sampled uniformly from 0 to 0.15.

Sinusoidal baseline drift is then added, with amplitude uniformly sampled from 0 to 0.3, frequency from 0.1 to 5 Hz, and phase from 0 to 2*pi.

Positive-class transient

For class 1, the waveform receives an additional Gaussian pulse:

```
g(u) = s*M*exp(-0.5*((u-p)/w)^2)
```

where u = sample_index/1024 is normalized waveform position, s is randomly -1 or +1, M is sampled uniformly from 0.2 to 1.5, p from 0.1 to 0.9, and w from 0.005 to 0.05.

The transient width is a fraction of waveform duration, not a value in seconds. The generator accepts a wider range of widths, but the dataset builder uses the narrower interval stated above.

Neural-network architecture

The classifier is a fully connected feedforward network with layer dimensions 1024-64-32-1. The two hidden layers use rectified linear unit activations. The output layer uses a sigmoid activation to estimate the positive-class probability.

Weights are initialized using a seeded standard normal distribution scaled by sqrt(2/fan_in), including the output layer [1]. All biases are initialized to zero.

The reference implementation uses NumPy float64 arithmetic and a numerically stable sigmoid evaluation.

Optimization

Training minimizes mean binary cross-entropy using manually implemented backpropagation and mini-batch stochastic gradient descent.

The learning rate is 0.01, the mini-batch size is 64, and training lasts 60 epochs. Initialization uses seed 2026. Each epoch uses a deterministic permutation generated with seed 3000 plus the zero-based epoch index.

Parameter updates follow:

```
parameter_new = parameter_old - learning_rate*mean_batch_gradient
```

The NumPy implementation computes gradients directly. The CUDA implementation uses PyTorch automatic differentiation and binary cross-entropy with logits.

Computational equivalence

The NumPy and CUDA training implementations use identical initialization, data, mini-batch order, learning rate, and float64 precision.

The implementations were compared through forward predictions, gradients, SGD updates, individual epochs, and complete 60-epoch training runs. The recorded 61-checkpoint learning curves agreed within a maximum absolute metric difference of 2.2204460492503131e-16.

Both experiment harnesses use the NumPy forward pass and shared evaluation utilities to calculate recorded training and validation metrics. Therefore, the learning-curve comparison is not an independent verification of the evaluation-metric implementation.

Performance experiments

Exploratory benchmarks measured repeated CPU and CUDA training epochs after warmups. Experiments varied dataset size, CUDA mini-batch size, and CUDA floating-point precision.

The reference configuration uses float64 precision and batch size 64. CUDA timing includes tensor setup and output transfer unless otherwise stated.

Reported measurements characterize these implementations on the tested hardware; they do not establish general CPU-versus-GPU performance superiority.

Evaluation and checkpoint selection

The classification threshold was fixed at 0.5. ROC-AUC was calculated using average ranks, with tied positive-negative score pairs receiving half credit [4]. The evaluation implementation agreed with an independent pairwise reference across 104 synthetic verification cases.

The primary model was selected at epoch 60 for the highest recorded validation ROC-AUC, with earliest epoch breaking ties. A secondary epoch-48 checkpoint minimized validation loss. Both checkpoints reproduced archived validation metrics before the held-out evaluation.

A final evaluation protocol was approved and committed before the held-out test was accessed. The epoch-60 primary model was then evaluated once without test-informed model selection, retraining, or threshold adjustment.

3. Results

3.1 Final held-out classification

The primary model was evaluated on 1,800 balanced synthetic signals.

Metric	Held-out value
Accuracy	78.78%
Precision	79.37%
Recall	77.78%
F1	0.7856
ROC-AUC	0.8613
Binary cross-entropy	0.6112
True positives	700
True negatives	718
False positives	182
False negatives	200

Among 900 transient-containing waveforms, 700 were correctly detected and 200 were missed. Among 900 background waveforms, 718 were correctly rejected and 182 generated false alarms.

Accuracy decreased from 79.56% on validation to 78.78% on the held-out test, a decline of 0.78 percentage points. ROC-AUC declined from 0.8742 to 0.8613, while binary cross-entropy increased from 0.5745 to 0.6112.

Training accuracy reached approximately 99.75%, substantially exceeding held-out accuracy. This generalization gap indicates that the learned training performance did not fully transfer to independently generated examples.

Evidence: `results/heldout_final_primary_v1.json` and `docs/FINAL_RESULTS.md`.

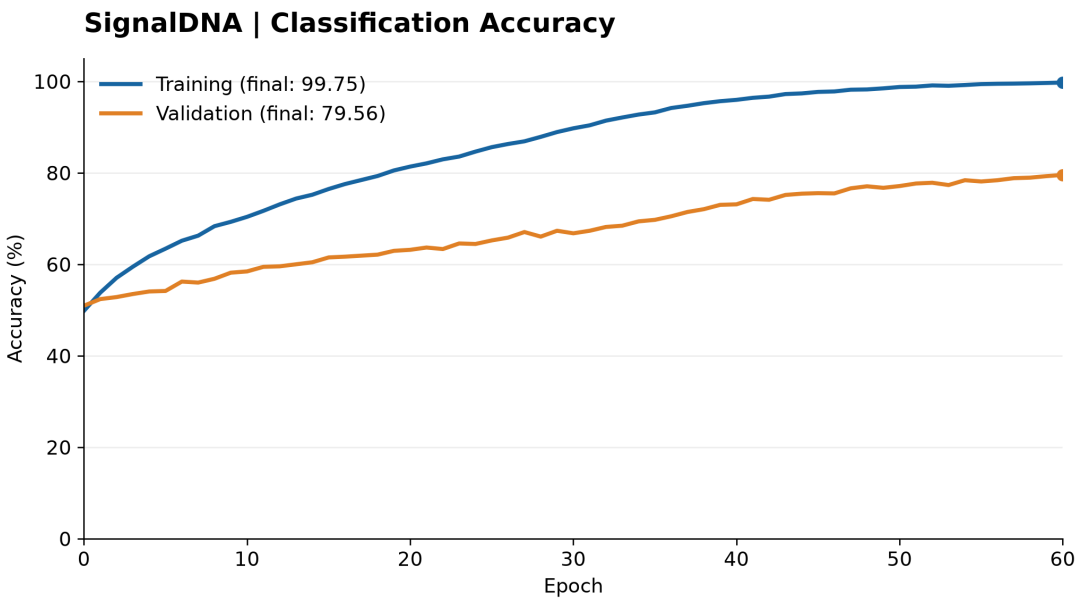


Figure 1. Training and validation accuracy across the archived 60-epoch NumPy run.

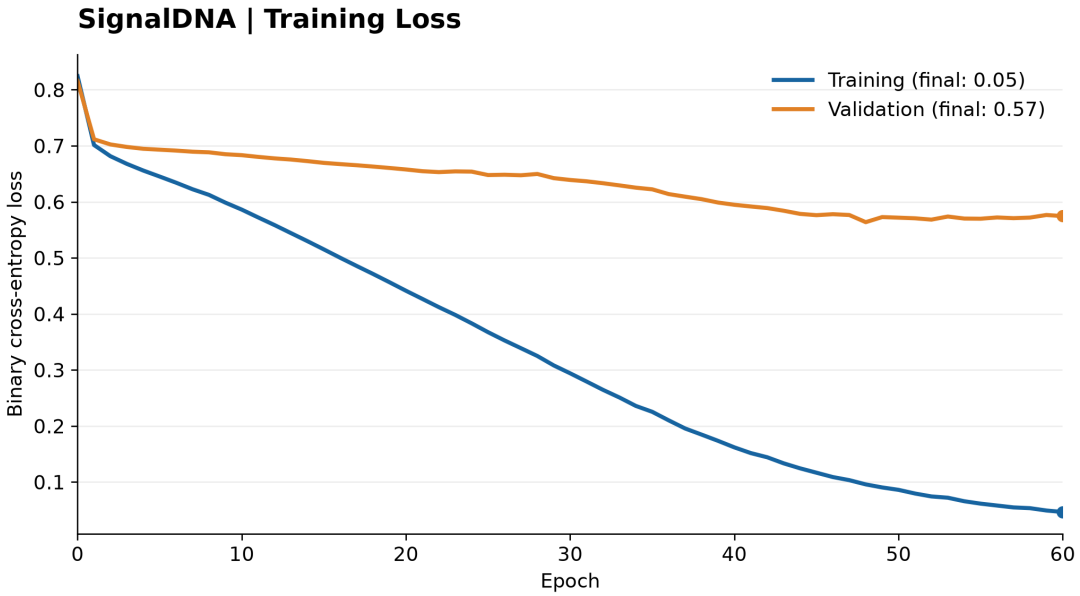


Figure 2. Training and validation binary cross-entropy; divergence illustrates the generalization gap.

3.2 Numerical reproducibility

The CPU and CUDA implementations agreed across all 61 recorded training checkpoints. The maximum absolute metric difference was 2.2204460492503131e-16. Additional comparisons covered forward propagation, gradients, SGD updates, and training epochs.

Both training harnesses used the common NumPy evaluation implementation for logged metrics. Agreement of those logged metrics does not constitute an independent verification of every evaluation calculation.

3.3 CPU/GPU performance

The five-run warmed-up benchmark produced median epoch times of approximately 98.7 ms for CPU and 167.4 ms for CUDA. CUDA was approximately 1.70 times slower for this configuration.

CPU execution was faster at all four measured dataset sizes using float64 and batch size 64. Increasing the CUDA batch size reduced measured epoch time but changed the number of optimization updates.

CUDA float32 execution was slower than float64 in the tested precision experiments, including alternating-order measurements. A change to PyTorch matrix-multiplication precision settings did not eliminate the observed slowdown. These measurements do not establish the precise underlying kernel-level cause.

The archived JSON benchmark records contain the supporting observations.

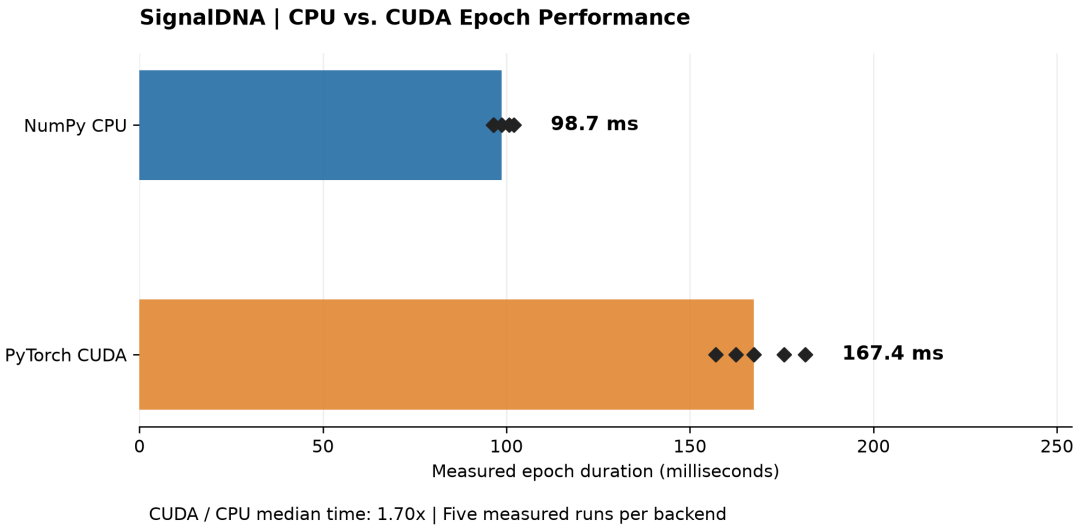


Figure 3. Five warmed-up epoch measurements per implementation; bars show medians and diamonds individual runs. Float64, batch size 64; workload-specific observations.

4. Discussion

The experiments demonstrate reproducible training behavior across the two tested implementations and predictive discrimination on independently generated synthetic signals.

The classification results are encouraging but imperfect. False alarms, missed transients, and the large training-to-test gap indicate substantial room for improvement.

The computational results also challenge a simplistic assumption that a GPU must outperform a CPU. Performance depends on workload size, numerical precision, launch and transfer overhead, implementation details, and optimization strategy.

A major practical lesson is the importance of verification discipline: comparing independent implementations, maintaining immutable evidence, separating validation from held-out testing, and preserving unexpected negative results.

PyTorch documents asynchronous CUDA execution and the need for synchronization or CUDA events when measuring device execution time [3]. Benchmark timing boundaries must be reviewed against the archived scripts before publication.

5. Limitations

The dataset is entirely synthetic and represents a constrained family of waveform-generating processes. The findings do not establish performance on real sensor recordings, changing acquisition conditions, or unfamiliar noise distributions.

The study used one principal network architecture, one principal training seed, and a single final held-out evaluation. Cross-seed uncertainty and confidence intervals were not established. Validation data was consulted repeatedly during exploratory development.

Performance findings apply only to the tested hardware, software, batch sizes, and workload configurations. No FPGA inference implementation, embedded deployment, power measurement, or hardware-design productivity improvement has been demonstrated.

6. Reproducibility and Evidence

The repository contains source code, experiments, checkpoints, archived JSON results, and a research evidence register at `results/research_evidence_register_v1.json`.

The final primary checkpoint and held-out results have recorded SHA-256 fingerprints. The evaluation attempt was locked against automatic repetition.

Before public release, environment documentation, executable reproduction instructions, third-party licensing, repository hygiene, and external references require review.

7. Future Work

Possible next steps include stronger classical signal-processing baselines, cross-seed analysis on development data, regularization experiments, failure-case analysis, and investigation using representative real-world sensor signals.

Future embedded and FPGA experiments may investigate quantization, inference latency, resource utilization, power consumption, streaming data paths, and whether neural networks provide measurable improvements over established engineering methods.

Any work informed by the completed held-out evaluation must be identified as exploratory rather than another untouched final evaluation.

8. Conclusion

SignalDNA provided an independently implemented neural-network reference, closely matching CPU/CUDA training results, and a documented held-out assessment of synthetic transient detection. The primary model reached 78.78% held-out accuracy and 0.8613 ROC-AUC while revealing substantial generalization and performance limitations.

Beyond these numerical results, the project established a reproducible learning and experimentation process. It is an initial foundation for further investigation of neural networks in embedded systems and FPGA engineering, not proof of benefits in those applications.

References

[1] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. Proceedings of ICCV, pp. 1026-1034. https://openaccess.thecvf.com/content_iccv_2015/html/He_Delving_Deep_into_ICCV_2015_paper.html

[2] NumPy Developers. numpy.random.SeedSequence. https://numpy.org/doc/stable/reference/random/bit_generators/generated/numpy.random.SeedSequence.html

[3] PyTorch Contributors. CUDA Semantics. https://docs.pytorch.org/docs/stable/notes/cuda.html

[4] scikit-learn Developers. roc_auc_score. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html